

StockWise

About the Project

- StockWise is a smart inventory and stock tracking system.
- It tracks stock levels in real-time and sends alerts when stock runs low.
- It shows inventory data on a dashboard with charts and tables.
- Built for small and medium-sized businesses.

- Zehra Berre Akyuz — Project Manager
- Tuba Eraslan — Backend
- Ceren Sengul — Frontend
- Can Tasar — QA & Documentation

1. Risk Identification

Risk	Probability	Impact	Risk Level
Data Loss	Medium	High	HIGH
Security Breach	Low	High	MEDIUM
System Performance Issues	Medium	Medium	MEDIUM
Integration Issues	Medium	Medium	MEDIUM
User Errors	High	Medium	MEDIUM
Server / Deployment Failures	Low	High	MEDIUM

1.1 Risk Assessment & Management

- Each risk is evaluated based on two factors: Probability and Impact.
- High probability + high impact risks are handled first.
- The team monitors risks continuously during the 14-week project.

1.2 Mitigation / Preventive Actions

- Data Loss — Regular database backups and a disaster recovery plan.
- Security — Role-based access control, strong password hashing, and HTTPS encryption.
- Performance — Database query optimization and load testing.
- Integration — API documentation and testing between frontend and backend.
- User Errors — Confirmation prompts for critical actions and audit logs.
- Server Failures — Reliable hosting, automated deployment, and a rollback plan.

2. Code Security & Data Privacy

- Users log in with a username and password. A secure token is created after login.
- Admins have full access. Staff have limited access. Permissions are checked on every action.
- Passwords are never stored as plain text. They are encrypted before saving.
- All data sent between the frontend and backend is encrypted.
- Database credentials and secret keys are never uploaded to GitHub.
- Changes to stock levels are logged automatically — we always know who changed what.

3. Implementation Roadmap

Phase	Key Tasks
Project Initiation	Team roles, Jira board, repository setup
Requirements Analysis	UML diagrams, risk identification, tech stack confirmed
System Design	DB schema (PostgreSQL), API specs, UI/UX design
Development	Backend APIs, frontend dashboard, unit testing
Testing & Validation	Integration tests, bug fixing, test reports
Final Delivery	Deployment, documentation, GitHub presentation

4. Validation & Testing Plan

Testing Type	What is Verified	Tool
Unit Testing	Service layer methods tested in isolation	JUnit 5 + Mockito
Integration Testing	API endpoints + frontend–backend flow	Postman + @SpringBootTest
Manual Testing	UI behavior, edge cases, role-based access	Browser + Jira
Security Testing	SQL injection, broken auth, access control	Postman (manual)
Performance Testing	API response time under expected load	Postman Runner

5. Success Criteria

Criterion	Target
Unit test coverage	≥ 80% of service layer methods
API response time	< 500ms for standard CRUD operations
JWT authentication	Unauthorized access correctly blocked